

```

import numpy as np
import matplotlib.pyplot as plt

# Activation Functions

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def relu(x):
    return np.maximum(0, x)

# Task 1
# Single Layer Neural Network

print("="*60)
print("TASK 1 : Single Layer Neural Network")
print("="*60)

# Input Vector
X = np.array([1, 2, 3])

# Weight Matrix (3 × 2)
W = np.array([
    [0.2, 0.5],
    [0.3, 0.7],
    [0.4, 0.9]
])

# Dot Product
Z = np.dot(X, W)

# Sigmoid Activation
Output = sigmoid(Z)

print("Input Vector:")
print(X)

print("\nWeight Matrix:")
print(W)

print("\nDot Product:")

```

```
print(Z)
```

```
print("\nOutput after Sigmoid:")
```

```
print(Output)
```

```
# Task 2
```

```
# Two Layer Neural Network
```

```
print("\n")
```

```
print("="*60)
```

```
print("TASK 2 : Two Layer Neural Network")
```

```
print("="*60)
```

```
# Input Layer
```

```
X = np.array([1, 2, 3])
```

```
# Input → Hidden Weights
```

```
W1 = np.array([
```

```
    [0.2, 0.4, 0.1],
```

```
    [0.5, 0.3, 0.6],
```

```
    [0.7, 0.8, 0.9]
```

```
])
```

```
# Hidden → Output Weights
```

```
W2 = np.array([
```

```
    [0.3, 0.6],
```

```
    [0.4, 0.2],
```

```
    [0.5, 0.8]
```

```
])
```

```
# Hidden Layer
```

```
Z1 = np.dot(X, W1)
```

```
A1 = relu(Z1)
```

```
# Output Layer
```

```
Z2 = np.dot(A1, W2)
```

```
A2 = sigmoid(Z2)
```

```
print("Hidden Layer Activations:")
```

```
print(A1)
```

```
print("\nOutput Layer Activations:")
print(A2)
```

```
# Task 3
# Visualize Activations
```

```
print("\n")
print("="*60)
print("TASK 3 : Visualization")
print("="*60)
```

```
plt.figure(figsize=(12,4))
```

```
# Input
plt.subplot(1,3,1)
plt.bar(range(len(X)), X)
plt.title("Input Layer")
```

```
# Hidden
plt.subplot(1,3,2)
plt.bar(range(len(A1)), A1)
plt.title("Hidden Layer")
```

```
# Output
plt.subplot(1,3,3)
plt.bar(range(len(A2)), A2)
plt.title("Output Layer")
```

```
plt.tight_layout()
plt.show()
```

```
# Task 4
# Batch Forward Pass
```

```
print("\n")
print("="*60)
print("TASK 4 : Batch Forward Pass")
print("="*60)
```

```
np.random.seed(42)
```

```
# Batch of 3 inputs
X_batch = np.random.rand(3,4)

# Random Weights
W1 = np.random.rand(4,5)
W2 = np.random.rand(5,2)

# Hidden Layer
Hidden = relu(np.dot(X_batch, W1))

# Output Layer
Output = sigmoid(np.dot(Hidden, W2))

print("Batch Inputs:\n")
print(X_batch)

print("\nOutput Activations:\n")

for i in range(len(Output)):
    print(f"Input {i+1} -> {Output[i]}")
```

Explanation

Task 1 – Single Layer Neural Network

Step 1: Input Vector

$X = [1, 2, 3]$

This represents the input features.

Step 2: Weight Matrix

$W =$
[[0.2 0.5]
 [0.3 0.7]
 [0.4 0.9]]

Each column corresponds to one neuron in the output layer.

Step 3: Dot Product

$$Z = X \cdot W$$

This computes the weighted sum of the inputs.

Step 4: Sigmoid Activation

$$\text{Output} = \text{sigmoid}(Z)$$

The sigmoid function maps the weighted sums to values between **0 and 1**.

Task 2 – Two-Layer Neural Network

The data flows as:

Input Layer



Dot Product (W1)



ReLU



Hidden Layer



Dot Product (W2)



Sigmoid



Output Layer

Hidden Layer

$$Z1 = \text{np.dot}(X, W1)$$

$$A1 = \text{relu}(Z1)$$

ReLU keeps positive values and sets negative values to zero.

Output Layer

```
Z2 = np.dot(A1, W2)
```

```
A2 = sigmoid(Z2)
```

Sigmoid converts the outputs to probabilities.

Task 3 – Visualizing Activations

Three bar charts show how the data changes across the network:

- **Input Layer:** Original feature values.
- **Hidden Layer:** Activations after applying ReLU.
- **Output Layer:** Final values after applying Sigmoid.

This helps visualize how the neural network transforms the input data layer by layer.

Task 4 – Batch Forward Pass

Instead of processing one input at a time, a batch of three inputs is processed together.

```
X_batch.shape = (3,4)
```

This means:

- 3 input samples.
- Each sample has 4 features.

The forward pass is:

Batch Inputs (3×4)

|



Hidden Layer (3×5)

|



Output Layer (3×2)

Using NumPy matrix operations:

```
Hidden = relu(np.dot(X_batch, W1))
```

```
Output = sigmoid(np.dot(Hidden, W2))
```

The program prints the output activation for each of the three input vectors.